The Civic Dispatch

VOL. 1, NO. 046

2026-03-30

COVER STORY

Standards for ANSI escape codes

Hello! Today I want to talk about ANSI escape codes.

Julia Evans

For a long time I was vaguely aware of ANSI escape codes (“that’s how you make text red in the terminal and stuff”) but I had no real understanding of where they were supposed to be defined or whether or not there were standards for them. I just had a kind of vague “there be dragons” feeling around them. While learning about the terminal this year, I’ve learned that:

ANSI escape codes are responsible for a lot of usability improvements in the terminal (did you know there’s a way to copy to your system clipboard when SSHed into a remote machine?? It’s an escape code called OSC 52 !)

They aren’t completely standardized, and because of that they don’t always work reliably. And because they’re also invisible, it’s extremely frustrating to troubleshoot escape code issues.

So I wanted to put together a list for myself of some standards that exist around escape codes, because I want to know if they have to feel unreliable and frustrating, or if there’s a future where we could all rely on them with more confidence.

what’s an escape code?

ECMA-48

xterm control sequences

terminfo

should programs use terminfo?

is there a “single common set” of escape codes?

some reasons to use terminfo

some more documents/standards

why I think this is interesting

id=“what-s-an-escape-code”>what’s an escape code?

Have you ever pressed the left arrow key in your terminal and seen ^[[D ? That’s an escape code! It’s called an “escape code” because the first character is the “escape” character, which is usually written as ESC , \x1b , \E , \033 , or ^[.

Escape codes are how your terminal emulator communicates various kinds of information (colours, mouse movement, etc) with programs running in the terminal. There are two kind of escape codes:

input codes which your terminal emulator sends for keypresses or mouse movements that don’t fit into Unicode. For example “left arrow key” is ESC[D , “Ctrl+left arrow” might be ESC[1;5D , and clicking the mouse might be something like ESC[M :3 .

output codes which programs can print out to colour text, move the cursor around, clear the screen, hide the cursor, copy text to the clipboard, enable mouse reporting, set the window title, etc.

Now let’s talk about standards!

id=“ecma-48”>ECMA-48

ECMA-48 does two things:

Define some general formats for escape codes (like “CSI” codes, which are ESC[+ something and “OSC” codes, which are ESC] + something)

Define some specific escape codes, like how “move the cursor to the left” is ESC[D , or “turn text red” is ESC[31m . In the spec, the “cursor left” one is called CURSOR LEFT and the one for changing colours is called SELECT GRAPHIC RENDITION .

The formats are extensible, so there’s room for others to define more escape codes in the future. Lots of escape codes that are popular today aren’t defined in ECMA-48: for example it’s pretty common for terminal applications (like vim, htop, or tmux) to support using the mouse, but ECMA-48 doesn’t define escape codes for the mouse.

id=“xterm-control-sequences”>xterm control sequences

There are a bunch of escape codes that aren’t defined in ECMA-48, for example:

enabling mouse reporting (where did you click in your terminal?)

bracketed paste (did you paste that text or type it in?)

OSC 52 (which terminal applications can use to copy text to your system clipboard)

I believe (correct me if I’m wrong!) that these and some others came from xterm, are documented in XTerm Control Sequences , and have been widely implemented by other terminal emulators.

This list of “what xterm supports” is not a standard exactly, but xterm is extremely influential and so it seems like an important document.

id=“terminfo”>terminfo

In the 80s (and to some extent today, but my understanding is that it was MUCH more dramatic in the 80s) there was a huge amount of variation in what escape codes terminals actually supported.

To deal with this, there’s a database of escape codes for various terminals called “terminfo”.

It looks like the standard for terminfo is called X/Open Curses , though you need to create an account to view that standard for some reason. It defines the database format as well as a C library interface (“curses”) for accessing the database.

For example you can run this bash snippet to see every possible escape code for “clear screen” for all of the different terminals your system knows about:

for term in $(toe -a | awk ‘{print $1}’) do echo $term infocmp −1 -T “$term” 2>/dev/null | grep ‘clear=’ | sed ‘s/clear=//g;s/,//g’ done

On my system (and probably every system I’ve ever used?), the terminfo database is managed by ncurses.

id=“should-programs-use-terminfo”>should programs use terminfo?

I think it’s interesting that there are two main approaches that applications take to handling ANSI escape codes:

Use the terminfo database to figure out which escape codes to use, depending on what’s in the TERM environment variable. Fish does this, for example.

Identify a “single common set” of escape codes which works in “enough” terminal emulators and just hardcode those.

Some examples of programs/libraries that take approach #2 (“don’t use terminfo”) include:

kakoune

python-prompt-toolkit

linenoise

libvaxis

chalk

I got curious about why folks might be moving away from terminfo and I found this very interesting and extremely detailed rant about terminfo from one of the fish maintainers , which argues that:

[the terminfo authors] have done a lot of work that, at the time, was extremely important and helpful. My point is that it no longer is.

I’m not going to do it justice so I’m not going to summarize it, I think it’s worth reading.

id=“is-there-a-single-common-set-of-escape-codes”>is there a “single common set” of escape codes?

I was just talking about the idea that you can use a “common set” of escape codes that will work for most people. But what is that set? Is there any agreement?

I really do not know the answer to this at all, but from doing some reading it seems like it’s some combination of:

The codes that the VT100 supported (though some aren’t relevant on modern terminals)

what’s in ECMA-48 (which I think also has some things that are no longer relevant)

What xterm supports (though I’d guess that not everything in there is actually widely supported enough)

and maybe ultimately “identify the terminal emulators you think your users are going to use most frequently and test in those”, the same way web developers do when deciding which CSS features are okay to use

I don't think there are any resources like Can I use...? or Baseline for the terminal though. (in theory terminfo is supposed to be the "caniuse" for the terminal but it seems like it often takes 10+ years to add new terminal features when people invent them which makes it very limited)

id="some-reasons-to-use-terminfo">some reasons to use terminfo

I also asked on Mastodon why people found terminfo valuable in 2025 and got a few reasons that made sense to me:

some people expect to be able to use the TERM environment variable to control how programs behave (for example with TERM=dumb), and there's no standard for how that should work in a post-terminfo world

even though there's less variation between terminal emulators than there was in the 80s, there's far from zero variation: there are graphical terminals, the Linux framebuffer console, the situation you're in when connecting to a server via its serial console, Emacs shell mode, and probably more that I'm missing

there is no one standard for what the "single common set" of escape codes is, and sometimes programs use escape codes which aren't actually widely supported enough

id="terminfo-user-agent-detection">terminfo & user agent detection

The way that ncurses uses the TERM environment variable to decide which escape codes to use reminds me of how webservers used to sometimes use the browser user agent to decide which version of a website to serve.

It also seems like it's had some of the same results – the way iTerm2 reports itself as being "xterm-256color" feels similar to how Safari's user agent is "Mozilla/5.0 (Macintosh; Intel Mac OS X 14_7_4) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.3 Safari/605.1.15". In both cases the terminal emulator / browser ends up changing its user agent to get around user agent detection that isn't working well.

On the web we ended up deciding that user agent detection was not a good practice and to instead focus on standardization so we can serve the same HTML/CSS to all browsers. I don't know if the same approach is the future in the terminal though – I think the terminal landscape today is much more fragmented than the web ever was as well as being much less well funded.

id="some-more-documents-standards">some more documents/standards

A few more documents and standards related to escape codes, in no particular order:

the Linux console_codes man page documents escape codes that Linux supports

how the VT 100 handles escape codes & control sequences

the kitty keyboard protocol

OSC 8 for links in the terminal (and notes on adoption)

A summary of ANSI standards from tmux

this terminal features reporting specification from iTerm

sixel graphics

id="why-i-think-this-is-interesting">why I think this is interesting

I sometimes see people saying that the unix terminal is "outdated", and since I love the terminal so much I'm always curious about what incremental changes might make it feel less "outdated".

Maybe if we had a clearer standards landscape (like we do on the web!) it would be easier for terminal emulator developers to build new features and for authors of terminal applications to more confidently adopt those features so that we can all benefit from them and have a richer experience in the terminal.

{

FRONT PAGE
Here’s a Standalone Cairo DLL for Windows

Jeff Preshing

Cairo is an open source C library for drawing vector graphics. I used it to create many of the diagrams and graphs on this blog.

Cairo is great, but it’s always been difficult to find a precompiled Windows DLL that’s up-to-date and that doesn’t depend on a bunch of other DLLs. I was recently unable to find such a DLL, so I wrote a script to simplify the build process for one. The script is shared on GitHub :

If you just want a binary package, you can download one from the Releases page:

The binary package contains Cairo header files, import libraries and DLLs for both x86 and x64. The DLLs are statically linked with their own C runtime and have no external dependencies. Since Cairo’s API is pure C, these DLLs should work with any application built with any version of MSVC. I configured these DLLs to render text using FreeType because I find the quality of FreeType-rendered text better than Win32-rendered text, which Cairo normally uses by default. FreeType also supports more font formats and gives text a consistent appearance across different operating systems.

id=“sample-application-using-cmake”>Sample Application Using CMake

Here’s a small Cairo application to test the DLLs. It uses CMake to support multiple platforms including Windows, MacOS and Linux.

Hope this helps somebody!

}

{

FEATURES

Smaller APKs with resource optimization

Jake Wharton

How many times does the name of a layout file appear in an Android APK? We can build a minimal APK with a single layout file to count the occurrences empirically.

Building an Android app with Gradle requires only one thing: an AndroidManifest.xml file with a package. From there we can add a dummy layout whose contents are just since we only care about its name.

Running gradle assembleRelease will produce a release APK measuring a paltry 2,118 bytes. We can dump its contents using xxd and look for home_view byte sequences.

class="highlight"> $ xxd build/outputs/apk/release/app-release-unsigned.apk ⁝ 000004c0: 0000 0074 0000 0018 0000 0072 6573 2f6c ...t.....res/l 000004d0: 6179 6f75 742f 686f 6d65 5f76 6965 772e ayout/home_view. 000004e0: 786d 6c63 66e0 6028 6160 6060 6490 61d0 xmlcf.`(a```d.a. ⁝ 00000570: 0000 0000 0000 0000 1818 7265 732f 6c61 ....res/la 00000580: 796f 7574 2f68 6f6d 655f 7669 6577 2e78 yout/home_view.x 00000590: 6d6c 0000 0002 2001 f801 0000 7f00 0000 ml.... ..... ⁝ 00000700: 0000 0000 0909 686f 6d65 5f76 6965 7700 ......home_view. 00000710: 0202 1000 1400 0000 0100 0000 0100 0000 ..... ⁝ 00000870: 0000 ad04 0000 7265 732f 6c61 796f 7574 ......res/layout 00000880: 2f68 6f6d 655f 7669 6577 2e78 6d6c 504b /home_view.xmlPK ⁝

There are three uncompressed occurrences of the path and one uncompressed occurrence of only the name in the APK based on this output.

If you have not read my post on calculating zip entry size or are not familiar with the structure of a zip file , a zip file is a list of file entries followed by a directory of all available entries. Each entry contains the file path and so does the directory. This accounts for the first occurrence (the entry header) and the last occurrence (the directory record) in the output.

The middle two occurrences in the output are from inside the resources.arsc file which is a database of sorts for resources. Its contents are visible because the file is uncompressed inside the APK. Running aapt dump –values resources build/outputs/apk/release/app-release-unsigned.apk shows the home_view record and its mapping to the path:

class="highlight"> Package Groups (1) Package Group 0 id=0x7f packageCount=1 name=com.example Package 0 id=0x7f name=com.example type 0 configCount=1 entryCount=1 spec resource 0x7f010000 com.example:layout/home_view: flags=0x00000000 config (default): resource 0x7f010000 com.example:layout/home_view: t=0x03 d=0x00000000 (s=0x0008 r=0x00) (string8) "res/layout/home_view.xml"

The APK contains a fifth occurrence of the name inside the classes.dex file. It does not show up in the xxd output because the file is compressed. Running baksmali dump

This is for the field inside the R.layout class which maps the layout name to a unique integer value. Incidentally, that integer is the index into the resources.arsc database to look up the associated file name for reading its XML contents.

To summarize the answer to our question, for each resource file, the full path appears three times and the name appears twice.

id="optimizing-resources">Optimizing resources

Android Gradle plugin 4.2 introduces the android.enableResourceOptimizations=true flag which will run optimizations targeted for resources. This invokes the aapt optimize command on the merged resources and resources.arsc file before they are packaged into the APK. The optimization only applies to release builds and will run regardless of whether minifyEnabled is set to true.

With the flag added to gradle.properties we can compare two APKs using diffuse to see its effects. The output is long, so we'll break it apart by section.

class="highlight"> | compressed | uncompressed ├──────────┼──────────┼──────────┼──────────┼──────────┼──────────┤ APK | old | new | diff | old | new | diff ──────────┼──────────┼──────────┼──────────┼──────────┼──────────┤ dex | 695 B | 695 B | 0 B | 1,016 B | 1,016 B | 0 B arsc | 682 B | 674 B | −8 B | 576 B | 564 B | −12 B manifest | 535 B | 535 B | 0 B | 1.1 KiB | 1.1 KiB | 0 B res | 185 B | 157 B | −28 B | 116 B | 116 B | 0 B asset | 0 B | 0 B | 0 B | 0 B | 0 B | 0 B other | 22 B | 22 B | 0 B | 0 B | 0 B | 0 B ──────────┼──────────┼──────────┼──────────┼──────────┼──────────┤ total | 2.1 KiB | 2 KiB | −36 B | 2.7 KiB | 2.7 KiB | −12 B

First is a diff of the contents in the APK. The "compressed" columns are the size cost inside the APK, and the "uncompressed" columns are the cost when extracted.

The res category represents our single resource file whose size dropped 28 bytes. The arsc category is for the resource.arsc file which itself dropped 8 bytes. We'll see the cause of these changes shortly.

class="highlight"> DEX | old | new | diff ──────────┼──────────┼──────────┼──────────┤ files | 1 | 1 | 0 strings | 15 | 15 | 0 (+0 −0) types | 8 | 8 | 0 (+0 −0) classes | 2 | 2 | 0 (+0 −0) methods | 3 | 3 | 0 (+0 −0) fields | 1 | 1 | 0 (+0 −0)

ARSC | old | new | diff ──────────┼──────────┼──────────┼──────────┤ configs | 1 | 1 | 0 entries | 1 | 1 | 0

These two sections represent the code and contents of the resource database. Having no changes, we can infer that the optimizations have not affected the R.layout.home_view field nor the home_view resource entry.

```
}
{
```

ActionBarSherlock - A Love Story (Part 3)

Jake Wharton

I am talking, of course, about version 3 and version 4, respectively. And I'm also lying a bit because I won't just be abandoning the 3.x users either. I'll give you a two-month deprecation window from version 4's release. Because...

ActionBarSherlock v4 is coming and it is awesome.

Now I realize that I am a bit biased, but let me explain how this version is the first version that I think I will be truly proud of.

No more shuffling between native and custom implementations.

Google's support library operates in this way in that it makes no attempt to use any native implementations even if they exist. It is far easier and more stable to keep all of the functionality in the library. Plus, the Android 4.0 action bar has been designed to accommodate every conceivable screen size that the platform can run on so why should we continue bothering to switch to the native implementation?

Additionally, this change became more and more of an apparent need rather than a choice due to changes in Android 4.0's MenuItem interface.

The support library classes are no longer included in the core of the library.

Though somewhat ironic based on the last point, the decision to allow the library to stand alone was made in order to accommodate developers who were uncomfortable using a custom built version of the support library (or who even didn't use it at all).

A version of the support library will be provided as a plugin .jar that has been modified to add ActionBarSherlock support. The changes to the library will be kept at a minimum and will not include any unrelated fixes. File bugs on b.android.com for that, please.

WARNING: This means that if you are using FragmentMapActivity or using fragments in SherlockPreferenceActivity you WILL have to change your implementation or create your own versions of these base classes. I will no longer be maintaining support for these.

Extending from a custom base activity is no longer required (but still recommended).

Similar to how ActionBarSherlock v1 and v2 operated, you can perform static attachment of the action bar to your activities. This allows for the use of alternate base activities such as those provided by other third-party libraries (e.g., RoboGuice).

The added side-effect of this is that all of the interaction logic has been placed within this single class which is also the one used by the base activities. This means that whether you do use a base activity or choose to interact with the static attachment you are afforded the full API.

Fully mirrored theming support to mimic the native action bar.

Forget the 'ab'-prefixed attributes of v3.x, v4 now allows for defining proper styles for the action bar, action mode, and various other sub-components of the action views.

It is the Ice Cream Sandwich action bar!

...but you probably knew that already.

Split action bar, action modes, action providers, condensed tab navigation, and so much more!

I have been working on this for nearly 8 weeks now so it's easy for me to get excited. Starting tomorrow the version 4 beta will be officially announced and detailed in a much more technical manner so that you can begin testing and hopefully join in the excitement.

As it stands now there are still large bugs and "bugs" with version 4. You can find them under milestone 4.0.0 on the GitHub issue tracker. As always, code contributions are welcomed and encouraged.

There is no timeline yet for the final release. There will be one or two release candidates before which is when I will be working with a few devs on real implementations to determine any problems that exist. If everything goes smoothly the final release will not be far behind that.

Thank you everyone for your support thus far. Happy new year to all.

```
}
{
```

Multiplatform Compose and Gradle module metadata abuse

Jake Wharton

My primary work project for the better part of a year (named Redwood) is built on top of Compose 1 and runs on every platform that Kotlin supports. This of course means Android, but we also have Compose running on iOS, the web, the JVM, and all other native targets. It's truly a multiplatform Compose project 2 .

Getting Compose to run on all these platforms isn't as hard as you would think. The Compose runtime is written as multiplatform Kotlin code but Google only ships it compiled for Android. JetBrains goes farther by shipping versions compiled for the web and for the JVM. We simply go the whole distance and compile it for every Kotlin target, while also shipping it as a single Kotlin multiplatform artifact.

For a year this worked fine. However, Compose UI recently went stable which meant our Android engineers were eager to start using it in the main app (as opposed to just samples). Upon Compose UI's introduction D8 fails with a duplicate class error:

The androidx.compose.* types are compiled into Redwood's multiplatform Compose runtime artifact. Compose UI depends on the official Compose runtime for Android which also contains these types. Since the two artifacts have different Maven coordinates, Gradle allows both to be included in the app which eventually causes D8 to complain 3 .

Redwood was already building Compose from the same git SHAs as Google's release builds. Ideally we could use our own builds for every platform except Android, and then point at Google's artifact solely for Android. This would allow Gradle to see the two projects as sharing a common dependency thereby de-duplicating the Compose runtime classes.

id="gradle-module-metadata">Gradle module metadata

The mechanism by which Kotlin multiplatform artifacts resolve the correct dependency is through Gradle's module metadata format .

Gradle Module Metadata is a unique format aimed at improving dependency resolution by making it multi-platform and variant-aware.

The module metadata is a JSON document which describes the supported platforms through key/value attributes. For Redwood's Compose runtime the module metadata looks roughly like this:

“component” : {

“group” : “app.cash.redwood” ,

“module” : “compose-runtime” ,

“version” : “0.1.0-square.15”

},

“variants” : [

{

“name” : “releaseApiElements-published” ,

“attributes” : {

“org.gradle.usage” : “java-api” ,

“org.jetbrains.kotlin.platform.type” : “androidJvm”

},

“available-at” : {

“url” : “../../compose-runtime-android/0.1.0-square.15/compose-runtime-android-0.1.0-square.15.module” ,

“group” : “app.cash.redwood” ,

“module” : “compose-runtime-android” ,

“version” : “0.1.0-square.15”

}

},

{

“name” : “iosArm64ApiElements-published” ,

“attributes” : {

“artifactType” : “org.jetbrains.kotlin.klib” ,

“org.gradle.usage” : “kotlin-api” ,

“org.jetbrains.kotlin.native.target” : “ios_arm64” ,

“org.jetbrains.kotlin.platform.type” : “native”

},

“available-at” : {

“url” : “../../compose-runtime-iosarm64/0.1.0-square.15/compose-runtime-iosarm64-0.1.0-square.15.module” ,

“group” : “app.cash.redwood” ,

“module” : “compose-runtime-iosarm64” ,

“version” : “0.1.0-square.15”

}

},

{

“name” : “[illegible]”,

“attributes” : {

“org.gradle.usage” : “java-runtime” ,

“org.jetbrains.kotlin.platform.type” : “androidJvm”

},

“available-at” : {

“url” : “../../compose-runtime-android/0.1.0-square.15/compose-runtime-android-0.1.0-square.15.module” ,

“group” : “app.cash.redwood” ,

“module” : “compose-runtime-android” ,

“version” : “0.1.0-square.15”

}

}

```
}
{
```

Play Services 5.0 Is A Monolith Abomination

Jake Wharton

Guava is a monolithic library, but that's not necessarily a bad thing. Nobody thinks twice when bundling it for the JVM. In the world of Android the mention of Guava has a bit of a negative stigma due to the dex file format's method limit and a concern about bloating APK size. The latter is no longer a valid argument. The dex method limit is a hard 64k limit to which Guava contributes just over 14k methods. 20% of this hard limit vanishes when you include Guava.

Sounds scary, right? It isn't.

Google Play Services 5.0 which just launched contributes over twenty thousand methods to your app. 20k+. One third of the limit! Now that is scary.

The Play Services library includes proprietary functionality built on the normal Android APIs and a separate APK downloaded on all devices with the Play Store. Some of the services it provides are invaluable. Like Guava it is also a monolothic library but it is a bad thing in this case.

A lot of really cool functionality is being put in Play Services. You'll have a hard time making a compelling app that lives in the Google Play ecosystem without it. You should want to put it in your applications and not have to worry about the overhead it brings.

Most of the library's offerings are very disparate, having only the fact that they're by Google as a common thread. This screams for small, modular artifacts which can be composed!

Google, it's time to unbundle. All the cool kids are doing it . (Spoiler alert: it happened)

At worst, we specify a few dependencies manually:

Best case would be a plugin that provided a clear DSL to what you were getting and offered easier configuration of the various components.

playServices { version '5.0.+' components 'ads' , 'analytics' , 'games' }

(You can even still provide the "fat" jar in both the dependency management world and the people who like manual dependency management.)

ProGuard is not the answer. Yes, for release builds it's nice to strip out any methods which are not being used. However, this is not justification for having large chunks of unused code as dependencies. Besides, if you read my post on a simulator you know that we deserve a faster development build pipeline which removes steps, not adds them.

It's not going to be a walk in the park but the packages inside Play Services are surprisingly well-configured to partitioning:

(Top-left: Games, top-center: Drive, middle-left: Plus, middle: common, middle-right: Maps, bottom: Ads)

Here's Guava for comparison which has less clear partition lines:

Here's how the method counts were determined:

$ curl 'http://search.maven.org/remotecontent?filepath=com/google/guava/guava/17.0/guava-17.0.jar' > guava.jar $ /android-sdk/build-tools/20.0.0/dx –dex –output guava.dex guava.jar $ dex-method-count guava.dex 14824

$ cp /android-sdk/extras/google/m2repository/com/google/android/gms/play-services/5.0.77/play-services-5.0.77.aar . $ unzip play-services-5.0.77.aar $ /android-sdk/build-tools/20.0.0/dx –dex –output play-services.dex classes.jar $ dex-method-count play-services.dex 20298

And the full by-package breakdown of Play Services:

$ dex-method-count-by-package play-services.dex

20298 com

20298 com.google

207 com.google.ads

169 com.google.ads.mediation

73 com.google.ads.mediation.admob

62 com.google.ads.mediation.customevent

20188 com.google.android

20188 com.google.android.gms

2 com.google.android.gms.actions

480 com.google.android.gms.ads

135 com.google.android.gms.ads.doubleclick

25 com.google.android.gms.ads.identifier

88 com.google.android.gms.ads.mediation

4 com.google.android.gms.ads.mediation.admob

73 com.google.android.gms.ads.mediation.customevent

26 com.google.android.gms.ads.purchase

118 com.google.android.gms.ads.search

866 com.google.android.gms.analytics

52 com.google.android.gms.analytics.ecommerce

10 com.google.android.gms.appindexing

```
}
{
```

Dockerizing a Rails application

Lazarus Lazaridis (iridakos)

Hey!

In this post we are going to:

create a docker image for the Rails chat application that we created in the previous post

configure the Docker environment and run the application by:

creating a container for the PostgreSQL database

creating a container for the Redis server

creating a container with the required configuration from the image we built

id="prerequisites">Prerequisites

id="install-docker">Install docker

The first thing you need in order to follow this tutorial is to install Docker on your machine.

We are going to use the Docker Community Edition . Follow the installation instructions matching your system.

I'm on Ubuntu 18.04 LTS so following the instructions , I had to:

Uninstall previous versions with:

I chose to install the application using the repository.

sudo apt-get install \ apt-transport-https \ ca-certificates \ curl \ gnupg-agent \ software-properties-common

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -

and I verified that I had the key with the proper fingerprint after executing:

I set up the repository with:

The installation took place with these commands:

sudo apt-get install docker-ce docker-ce-cli containerd.io

I checked that the installation was successful with:

Unable to find image 'hello-world:latest' locally latest: Pulling from library/hello-world 1b930d010525: Pull complete Digest: sha256:... Status: Downloaded newer image for hello-world:latest

Hello from Docker! This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.

(amd64)
3. The Docker daemon created a new container from that image which runs the

executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it

to your terminal.

To try something more ambitious, you can run an Ubuntu container with: $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID: https://hub.docker.com/

For more examples and ideas, visit: https://docs.docker.com/get-started/

id="important-note">Important note

Upon installation, a new user group was created with the name docker . If you want to allow users to do docker stuff without using sudo , you have to do some extra configuration (read more here) but keep in mind that:

The docker group grants privileges equivalent to the root user . For details on how this impacts security in your system, see Docker Daemon Attack Surface. – Post-installation steps for Linux - Official Docker installation instructions

We are going to use sudo in this tutorial.

id="clone-the-rails-chat-tutorial-from-github">Clone the rails chat tutorial from GitHub

Navigate to your development directory on your machine and clone the sample Rails chat application with:

Done.

id="configure-the-application-to-use-postgresql-for-the-production-environment">Configure the application to use PostgreSQL for the production environment

The application is configured to use the predefined sqlite database adapter. For the purpose of this tutorial we will change the adapter to postgresql and at the second part of the post we will create a container running the PostgreSQL database.

Open the file config/database.yml file and change the production configuration as described below:

Open the application's Gemfile and add the following lines:

to install the adapter. The pg gem requires to have the package libpq-dev installed on the machine. We will satisfy

}

standard-article(title: [Underlying structure and measurement invariance by sex of the state trait anxiety inventory: A psychometric analysis in Ecuador], author: [Alberto Rodríguez-Lorenzana], source-name: [PLOS ONE Feed], images: (), paragraphs: ([by Jose A. Rodas, Daniel Oleas, Guido Mascialino, José Alejandro Valdevila Figueira, Alberto Rodríguez-Lorenzana], [Anxiety is currently one of the most prevalent and investigated psychological symptoms requiring precise and reliable instruments for its assessment. The current study evaluates the psychometric properties of the State-Trait Anxiety Inventory (STAI) in an Ecuadorian sample, focusing on the factor structure and potential methodological artefacts, especially those introduced by reverse-scored items. Employing exploratory and confirmatory factor analyses (EFA and CFA), along with structural equation modelling (SEM), the research examines the presence of two distinct factors: genuine anxiety and a secondary factor related to reversed items. Findings indicate that the expected two-factor model fits the data more accurately than a unidimensional model. Notably, cross-loadings suggest the second factor may represent a substantive positive, anxiety-free state rather than solely a methodological effect. Additionally, multigroup CFA confirmed strict measurement invariance across sex, supporting the scale's utility for group comparisons. These results suggest that the STAI's total score represents a composite of anxiety and well-being. While the STAI remains a robust tool for evaluating anxiety in clinical contexts, researchers seeking pure measures of anxiety symptomatology should consider analysing the subscales separately.],), insert-map: (:), word-count: 192, edited-for-length: false, debug-mode: false,)

standard-article(title: [Book Review - Accelerate: The Science of Lean Software and Devops], author: [Dean Hume], source-name: [Dean Hume], images: (), paragraphs: ([During my morning coffee session, I like to check through my RSS feed and see what is happening out there in the world of technology. I came across this article by the Spotify R&D team entitled " Leveraging Mobile Infrastructure with Data-Driven Decisions ". They referenced the book Accelerate: The Science of Lean Software and Devops , and the title of the book instantly drew me in.], [I've just finished reading the book and I can say that I definitely enjoyed it.], [Through four years of research, the authors set out to find a way to measure software delivery performance—and what drives it—using rigorous statistical methods. This book presents both the findings and the science behind that research, making the information accessible for readers to apply in their own organizations. Personally, I think it's quite cool to see scientific data applied to some of the everyday things that we do in technology organizations. I also found it interesting to see how some of the top performing companies repeatedly seemed to pull away from the "pack" by simply applying many of the techniques applied in this book. The book takes a look at well-known best practices such as Continuous Delivery, Lean Management, and Transformational Leadership.], [If you are looking for a book that tells you how to build and scale a high performing technology organisation, then this book doesn't quite cover it. This book goes more into why you should be doing things a certain way and backs this up with facts.], [Part 1 of this book explores what the research team found after trawling through the data. Part 2 dives into the science behind the book and an introduction to Psychometrics. Finally Part 3, looks into transformation, which focuses on how leadership and management can help drive these improvements.], [If I had one takeaway from this book (which surprised me), it was that the leadership of an organisation plays a pivotal role in the team's results. I really liked this quote:], ["Leadership really does have a powerful impact on results. A good leader affects a team's ability to deliver code, architect good systems, and apply Lean principles to how the team manages its work and develops products. All of these have a measurable impact on satisfaction, efficiency, and the ability to achieve organisational goals".], [Overall, this is a great book and definitely gives a good insight into why some of the best practices that we take for granted are worth doing and lead to high performing technology organisations. This book concentrates more on the why as opposed to the how. If you are looking for a how to, I'd recommend reading The DevOps Handbook to learn more.], [I hope you enjoy the book!],), insert-map: (:), word-count: 453, edited-for-length: false, debug-mode: false,)

The book takes a look at well-known best practices such as Continuous Delivery, Lean Management, and Transformational Leadership.

Dean Hume

{

ANALYSIS

IN BRIEF

Rupak Ganguly, Serverless Blog: Build a serverless REST API service in Java, store the data in a DynamoDB table, and deploy it to AWS. All using the Serverless Framework.

Carlos Becker, Carlos Becker: Over the years I read several articles on how to be effective, and how the 10x engineer thing is or is not a lie and all that.

Fernando Medina Corey, Serverless Blog: Learn how to use the Serverless Framework to deploy your first Knative service on a Kubernetes cluster running in Google Cloud.

Carlos Becker, Carlos Becker: I have an old Couchbase 4.5.x cluster, and I thought it would be nice to upgrade it. These are my notes and the tests I did before doing it “in production”™.

Ben Thompson, Stratechery: Stratechery is on a bit of a disjointed Spring Break, as my usual week off will be spread out:

There will be no Update on Thursday, March 19

There will be no Update on Monday and Tuesday, March 23–24; there will be an Update and Interview on Wednesday and Thursday, March 25–26

There will be no Update on Monday, March 30

I will return to my usual posting schedule on Tuesday, March 31.

All other Stratechery Plus content, including my podcasts, will stay on schedule.

Carlos Becker, Carlos Becker: Since the infamous SolarWinds attack , supply chain integrity is something a lot of people are discussing and working on.

Gareth McCumskey, Serverless Blog: If all you want to do is play around and try stuff without worrying bills and costs, serverless is the place for you!

Gareth McCumskey, Serverless Blog: ShreyTheCray interviews Gareth McCumskey as he walks through the purpose, use cases, and process of getting started with the Serverless framework

Nelson Elhage, Nelson Elhage: Last time, I announced Check Plus, a declarative language for defining Check tests in C. This time, I want to talk about the tricks I used to implement a declarative minilanguage using the C preprocessor (and some GCC extensions). The Problem We want to write some toplevel declarations that look like: #define SUITE_NAME example BEGIN_SUITE(“Example test suite”); #define TEST_CASE core BEGIN_TEST_CASE(“Core tests”); … and so on, and somehow translate them into code that does the equivalent of:

}

The Civic Dispatch

Vol. 1, No. 046 · 2026-03-30

Curated and composed automatically.